

AADK 12: Task 2 — Animation Design & Planning in Compose

Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 12

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Research Animation APIs

animateFloatAsState

animateFloatAsState automatically animates a Float value whenever the target value changes, creating smooth transitions such as scaling or fading.

animateColorAsState

animateColorAsState smoothly transitions between colours when the target colour value changes.

AnimatedVisibility

AnimatedVisibility controls the appearance and disappearance of UI components with animation (fade, expand, slide, etc.).

animateContentSize

animateContentSize automatically animates layout size changes when the composable's content expands or shrinks.

Transition

A Transition coordinates multiple animations together when the UI changes between states.

Example Snippet (animateFloatAsState)

@Composable

```
fun AnimatedButton() {  
    var expanded by remember { mutableStateOf(false) }  
}
```

```

val scale by animateFloatAsState(
    targetValue = if (expanded) 1.2f else 1f
)

Button(
    onClick = { expanded = !expanded },
    modifier = Modifier.scale(scale)
) {
    Text("Tap Me")
}

```

This example enlarges the button slightly when it is tapped.

2. Selected App Feature for Animation

Feature: Favourite Animal Card Expansion (ZooTracker App)

When the user taps an animal card, the card expands to reveal additional information such as habitat, diet, and conservation status.

Animations improve the experience by:

- Making state changes visually clear and intuitive.
- Helping users understand relationships between UI elements without abrupt layout changes.

3. Animation Plan Document

Scenario	Trigger (User Action)	Animation Type	UI Elements Affected	Duration / Spec
Card Expand	Tap card header	animateContentSize() with spring	Card height, description text, image	300 ms spring

Scenario	Trigger (User Action)	Animation Type	UI Elements Affected	Duration / Spec
Favourite Toggle	Tap heart icon	animateColorAsState()	Heart icon colour	250 ms tween
Button Press Feedback	Tap button	animateFloatAsState()	Button scale	150 ms scale animation

4. Performance & Accessibility Notes

Performance Considerations

- Avoid heavy animations in large lists**
Animating many items in a LazyColumn can affect performance, so animations should be lightweight.
 - Limit recompositions**
Animations should only affect necessary UI elements to avoid unnecessary recomposition and UI lag.
-

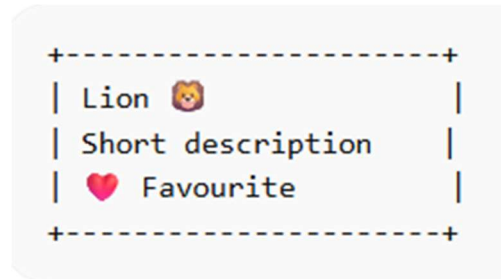
Accessibility & Motion Reduction

- Allow reduced motion**
Users with motion sensitivity should be able to disable or reduce animations in accessibility settings.
 - Ensure content remains readable**
Important information should remain visible even if animation is disabled or shortened.
-

5. Wireframe Sequence & Storyboard

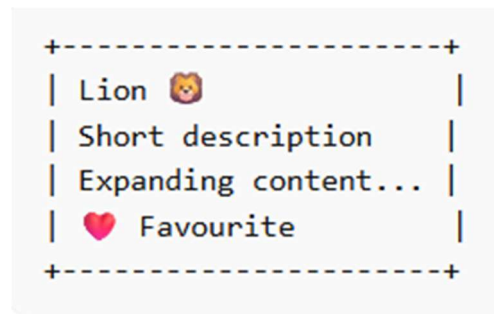
Scenario: Card Expansion Animation

Before State (Collapsed Card)



During Animation

- Card height increases
- Description fades in
- Image slightly expands



After State (Expanded Card)



Compose Tools Used

Compose Feature	Purpose
<code>animateContentSize()</code>	Smooth card expansion
<code>animateColorAsState()</code>	Heart icon colour change
<code>animateFloatAsState()</code>	Button scale feedback
<code>Modifier.graphicsLayer</code>	Control scale/alpha
<code>Modifier.alpha</code>	Fade content

Example usage:

```
Card(  
    modifier = Modifier  
        .fillMaxWidth()  
        .animateContentSize()  
)
```

6. Reflection

Designing animations before coding helps ensure that UI interactions feel intentional and meaningful rather than random. Planning animations allows developers to understand the states, transitions, and triggers clearly, improving both UX and code maintainability.

However, excessive animation can distract users and reduce performance, especially in complex interfaces. Overusing animations may also slow down navigation and make the interface feel cluttered.

Animations should only be used when they provide meaningful feedback or help guide user attention. If an animation does not improve clarity or usability, it is better to keep the interface simple and static.